Team Name: GatorStomp
Date: November 4, 2026

Pre-Alpha Build - GatorStomp

1 Introduction

There has been significant progress done for GatorStomp on the software and design side. The drivers and DSP are currently being worked on, where the team is building the process to communicate between the software. The OS foundation is being built, which will eventually encompass that software. The processor, PocketBeagleboard 2, has arrived and tested with. Further hardware like DAC and ADC are being researched and will be ordered soon.

2 Statement of Effort

2.1 Case Zumbrum

Setup the team wiki on Confluence, setup the team Jira board, setup the repository, setup the documentation structure for the project using Sphinx and Doxygen hosted on github here. Defined the packet structure for the audio orchestrator I/O system (issue). Built a script to help testing the I/O system of the audio ochestrator (issue). Built a simple version of the audio orchestrator for simple audio transformation (issue).

2.2 Shan Sundal

Designed the architecture for the input subsystem that will be a separate process which interacts with the audio orchestrator process. Researched rotary encoder and foot switch options and presented them to the team, and selected the components to order. Worked with Case to design and align on a message structure that will be used for communication between the input subsystem and the audio orchestrator. The full design, parts comparison, and message protocol, decision process, and difficulties/trade-offs with current decisions are documented on the team wiki on Confluence: Input Subsystem Design. Designed and implemented a mock audio orchestrator listener and PocketBeagle 2 button test that involved loading a default PocketBeagle Linux Debian and connecting to the button via GPIO: PR number 23 with this implementaiton.

Difficulties: the message layout changed twice during design (8 bytes, then 12, then the final 11), so the listener and test tools were written against `sizeof` the shared struct rather than a fixed number. When preparing to put the default Linux Debian on the PocketBeagle board, the login had to be set by editing `sysconf.txt` on the boot partition. This resolved an issue and made it possible to SSH to the board. Finding a usable button without soldering required reading the schematic, since the USER button only connects to boot mode pins, which was an unexpected difficulty.

2.3 Danush Singla

Completed building the foundation for the OS. Created Yocto layer that will contain the OS and built the first image. Researched and added Yocto layers that will be helpful and added as Kas system to help recover the code in the event it is lost. Each build took several hours, first was around 4 hours and second was 6-8 hours due to the Yocto layers being quite large to add. Also, contributed to the general CPU choice selection by researching possible CPUs and discussing with the team.

After OS was built, it was then edited to be made only for the Pocket Beagle 2. A USB driver was enabled along with SSH and DHCP server so that, when the USB-C cable is plugged into a laptop, that laptop can ssh into the board and work with the OS. This was tested and worked successfully. What was further tested was that compiled C code can be run on the OS. Furthermore, a debug log is used in the OS that lists every command that has ran on it, allowing for debug of later files.

Added instructions for Yocto use and other highly specific information to the team wiki on Confluence: OS Design.

2.4 Ajna Topić

Worked on developing the ADC/DAC software interface to allow communication with the audio orchestrator. Initially, design consisted of ALSA implementation with one process, and later expanded it to two processes to communicate with orchestrator through Linux pipes. Added functions that allow the orchestrator to directly read and write audio data through aforementioned pipes. Tested audio data transfer and documented implementation and testing on the team wiki.

3 Architecture

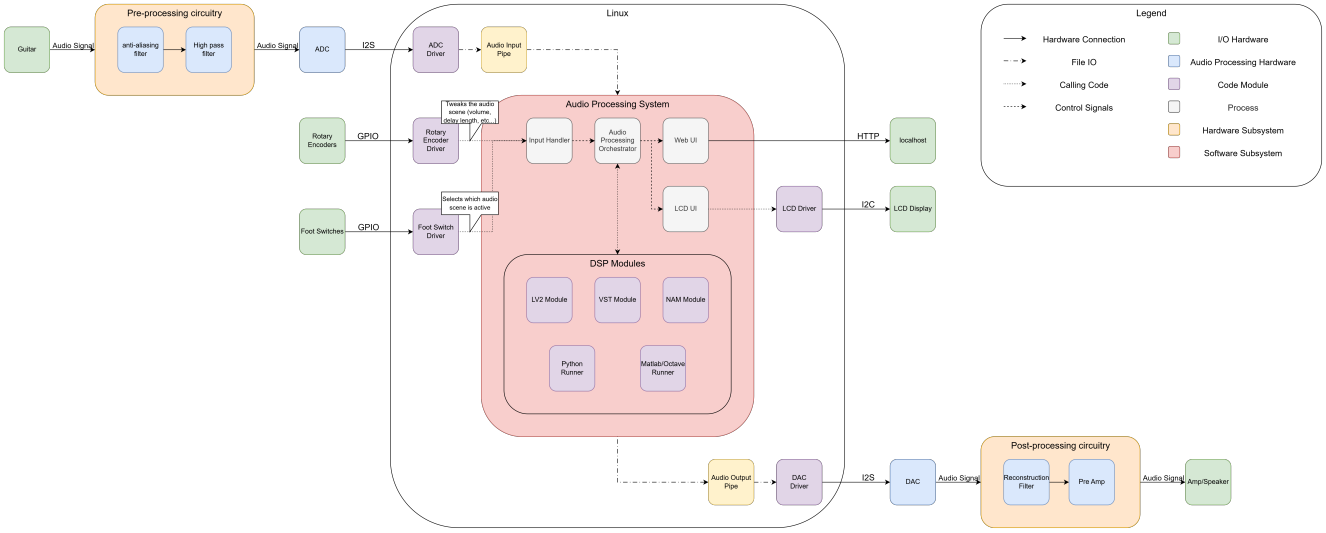


Figure 1: System Architecture

Figure 1 shows the current system architecture that we have designed. Each module is a CMake Project that will be spun up as a process on the operating system at boot using systemd. Pipes and sockets will be used to communicate between these processes.

3.1 External Interface

3.1.1 ADC and DAC

The external interface for ADC and DAC was implemented in C using ALSA (Advanced Linux Sound Architecture). The interface supports 96 kHZ, 24-bit stereo PCM audio. It reads audio into buffers and passes buffers to digital signal processing function (prototype for now). To test the implementation without physical hardware, Ubuntu WSL2 and ALSA virtual audio devices were used. A 440 Hz sine wave was generated using Python and passed through audio pipeline in 256 frame buffers.

The implementation was later expanded to use two separate processes, one for ADC capture and one for DAC playback. Two Linux pipes were implemented to allow communication between these processes and the audio orchestrator. The orchestrator can directly read audio data from the ADC pipe and write processed audio data to the DAC pipe. Four automated tests were developed and successfully passed using simulated PCM audio files to verify data integrity, pipe communication, and integration with the orchestrator.

3.1.2 Rotary Encoder and Foot Switch

The physical interface consists of four rotary encoders, which adjust effect parameters, and four momentary foot switches, which select the active scene. A separate user-space process, the input service (`gatorstomp-inputd`), reads them through the mainline Linux `rotary-encoder` and `gpio-keys` drivers and forwards each knob turn and switch press to the Audio Orchestrator over a Unix socket. Encoders send relative steps (usually $+1$ or -1) rather

than absolute positions, so the orchestrator owns the current value of every parameter.

The skeleton for this path is in place. The message format is defined in the shared header `packet_structure.h`, a mock listener stands in for the orchestrator, and three senders stand in for the input service: a scripted test sender, an interactive keyboard sender, and a GPIO sender that runs on the PocketBeagle 2 and uses the board's USER button as foot switch 0. The full chain (button, GPIO, sender, socket, listener) has been tested on the board.

The input subsystem connects to the rest of the system through the orchestrator: encoder messages will update the orchestrator's in-memory effect parameters, and foot switch messages will select the active scene. Saving scenes and parameters between boots is not yet implemented.

3.2 Persistent State and OS

Our system does not have a strong peristent state currently that is not tied to the OS. The audio transformation itself is ephemeral beyond the memory of the audio system, and we do not have any storage that persists between boots.

3.2.1 OS

The OS is built using Yocto, which provides the necessary tools to build the bare minimum necessary to make the GatorStomp system work. The OS itself wraps around the DSP logic and drivers on the PocketBeagleboard 2. It will perform functions like using systemd to startup necessary processes as quickly as possible, enabling drivers, and facilitating the communication between DSP and driver hardware through sockets. It should be noted that these features have not been added just yet. Additionally, existing Yocto layers provided for TI and ARM will be used to help map the connections between hardware on the PocketBeagleboard 2 and the software in the OS to streamline building.

In the event that something happens to the computer holding the OS code, it can be recovered using Kas. Kas is a tool that allows one, once they pull the repository, to build the entire OS and related Yocto layers so that they can continue working on the OS. Note: building the OS from scratch can take 6-8 hours at the very least. This recovery system was added in the commit hash on the repository: link to view commit

Furthermore, the OS was built with USB drivers so that a laptop plugged to it with only the USB-C cable can recognize the device. SSH and DHCP functionality written into the OS allows that laptop to ssh into the board and access the OS. Files can be transferred to it via sftp or other similar protocols. Some compiled C code was also ran on the OS in this way, confirming that code can be run on the OS without many issues. A debug file is also included that keeps a running tab on each command run on the OS, allowing for help to debug certain problems that may later appear when running code in the full system. This functionality is included in this commit: link to view commit

3.3 Internal Systems

We have implemented a simple version of the audio orchestrator that performs simple transforms on the audio being passed through the system. This system performs transformations on an input buffer to produce an output buffer. This system is a simple process that takes in the path to the input/output pipes and constantly performs it's transformation. In future implementations, what transformations are performed and full audio scenes will be controlled through the IO controller, but this is currently unimplemented. Note that this software has only been tested on simulations, and not on the actual board or OS.

4 Information Handling

Our current system does not have any data stored on the device constantly, only live data sent between processes.

4.1 Communication

The primary IPC at this time is between the audio orchestrator and the input service for the foot switches and rotary encoders. This uses a Unix `SOCK_SEQPACKET` socket at `/run/gatorstomp/orchestrator.sock`, where the

orchestrator is the server and the input service is the client. With systeminit, the orchestrator will create the socket at boot. Although the current implementation of the server does not act on the input it receives, the infrastructure for this is in place.

Every message, for the input subsystem, is the same packed 11 byte struct, `gs_socket_msg` (type, id, value, timestamp), defined once in a shared C++ header that both sides compile against, and mirrored in Python test tools as the `struct` format `<BBbq`. The input service sends `GS_MSG_BUTTON` for foot switches and `GS_MSG_TWEAK` for encoders. For testing, a C++ mock listener acts as the server, and Python scripts act as clients, including one that runs on the PocketBeagle 2 and reads a real button.

The architecture for the input and output pipes for audio has been defined and the pipes will act as temporary buffers for ADC readings and DAC outputs. The audio orchestrator will read from the input pipe and write to the output pipe. There will be processes that utilize the ADC/DAC drivers to fill/empty these pipes.

4.2 Integrity & Resilience

4.2.1 Input Subsystem (Rotary Encoders and Foot Switches)

The following measures are implemented:

- The socket type, `SOCK_SEQPACKET`, preserves message boundaries, so messages cannot arrive merged or split.
- A compile-time `static_assert` in the shared header fails the build if the struct is ever not exactly 11 bytes, for example if the `packed` attribute is removed or a field changes.
- The mock listener rejects and logs messages with the wrong size (using `MSG_TRUNC` so oversized messages are caught too), an unexpected type, an id outside 0 to 3, or a foot switch value other than 0 or 1, and keeps running.
- The mock listener goes back to waiting when a client disconnects, and removes a stale socket file left by a crash before starting.
- The GPIO sender on the board filters switch bounce with a 10 ms debounce.

The following measures are planned:

- The real orchestrator will apply the same message checks as the mock listener.
- The input service will reconnect with backoff if the orchestrator restarts, and re-send the foot switch states after reconnecting.
- Mechanical switch bounce on the real controls will be filtered by the `gpio-keys` debounce interval and the RC filter circuit recommended in the encoder datasheet.
- Only trusted processes should be able to send input: the socket will live in a root-owned `/run/gatorstomp/` directory with restricted permissions, and the orchestrator can check the connecting process with `SO_PEERCRED`.

Known issue: message timestamps use wall clock time, but the PocketBeagle 2 has no battery-backed clock or network time over USB, so timestamps are wrong after every boot. A resolution for this is being planned (likely where the current time will be pulled from the device the board is connected to).

Currently we do not have a plan for byte misalignment issues with the audio input/output pipes, this will be handled when we have a more fleshed out audio orchestrator.

As of right now, the OS does not have any form of protection against unauthorized access. Eventually, access will be protected using some form of ssh authentication when everything is near production.

5 Project Documentation

Documentation for the project is provided on three different layers: the GitHub README, Sphinx Documentation, and a team wiki on Confluence. The README will contain the most minimal amount of documentation regarding how to build and setup the project. Sphinx contains documentation for the code and functionality of certain processes for drivers. Certain specific information like how many bits are used by the Audio Orchestrator are

included in these to help establish protocols. Finally, the team wiki is used for almost everything. The wiki includes code documentation, team decisions, and general ideas and structure for parts of the project.

5.1 Important Links

- GitHub Repository
- Hosted Documentation
- Confluence Team Wiki
- Jira Kanban Board